Paperscope: Embedding-Powered Analysis Tools for Academic Papers

Ian Todd
Sydney Medical School
University of Sydney
Sydney, NSW, Australia
itod2305@uni.sydney.edu.au
ORCID: 0009-0002-6994-0917

March 2026

Abstract

Paperscope is a Python toolkit that applies sentence-level embedding analysis to academic papers during the writing process. Given a LaTeX manuscript and a corpus of reference texts, it embeds both into a shared vector space and measures distances to answer questions that previously required manual reading: do citations actually support the claims they are attached to? Which claims are most novel? What would a reviewer likely object to? Is the abstract covering all sections? Which journals are the best semantic fit? The toolkit includes 12 analysis tools, a bibliography management pipeline, and a literature discovery system, all designed to integrate with AI coding assistants such as Claude Code and Codex. Paperscope is open-source under the MIT license and available at https://github.com/todd866/paperscope.

1 Introduction

AI-assisted academic writing is accelerating. Large language models can draft manuscripts, suggest framings, and identify relevant literature faster than manual processes. But this acceleration creates a quality-control bottleneck: the speed at which text is produced outpaces the speed at which its scholarly integrity can be verified.

Three specific failure modes arise in AI-assisted research writing:

1. **Citation misalignment.** A sentence cites a reference for a claim that the reference does not actually support. This is the most common failure mode of LLM-assisted drafting — the model generates plausible-sounding citations that are semantically adjacent to, but not actually about, the claim being made.

2. **Undetected novelty.** The author cannot easily tell which claims in their paper are genuinely new (requiring careful justification) and which are well-supported by existing literature (requiring only a citation). This distinction determines how reviewers will respond.

3. **Blind spots in coverage.** The bibliography may be missing highly relevant work, or the abstract may fail to represent major sections of the paper. These gaps are invisible to the author during writing but immediately apparent to reviewers.

Paperscope addresses these problems by embedding the paper and its literature into a shared vector space and measuring distances. The approach is simple — cosine similarity over sentence-transformer embeddings — but it enables analyses that are otherwise prohibitively tedious.

2 Architecture

Paperscope has three layers, each usable independently (Figure 1).

```
LaTeX project → bib/ → bibliography.json → DOI resolution
OpenAlex/arXiv → harvest/ → PDF download → text extraction
Paper + literature → text/ → embed/ → analysis/ → results
```

Figure 1: Three-layer architecture. Each layer feeds the next but can be used standalone.

Layer 1: Bibliography management (`bib/`). Extracts citations from `.tex` and `.bib` files, resolves missing DOIs via the CrossRef API [CrossRef, 2024], and verifies resolved DOIs against metadata. This catches a common AI-assisted writing failure: hallucinated or misattributed references.

Layer 2: Literature ingestion (`harvest/`, `ingest/`). Discovers new papers from OpenAlex [Priem et al., 2022], arXiv, and bioRxiv matching a configurable research profile. Downloads open-access PDFs via Unpaywall, extracts text using PyMuPDF, and stores extracted text in the repository alongside the manuscript.

Layer 3: Embedding analysis (`text/`, `embed/`, `analysis/`). The core contribution. LaTeX is cleaned to plain text, split into overlapping chunks (∼200 words, 50-word overlap), and encoded using sentence-transformers [Reimers and Gurevych, 2019] with the `all-MiniLM-L6-v2` model (384 dimensions). Reference texts are chunked and embedded identically. All analysis tools operate on the resulting cosine similarity matrices.

2.1 Text Processing

The `text/` module provides three shared utilities. `clean_latex()` strips LaTeX commands, math environments, URLs, and special characters via iterative regex unwrapping (up to 6 nesting levels). `chunk_text()` splits text into overlapping word-level chunks using a step-based approach: each chunk starts `step = target − overlap` words after the previous one. `extract_citation_contexts()` finds sentences containing `\cite{}` commands, returning the cleaned text, cited keys, and line numbers for traceability.

2.2 Embedding with Fallback

The `embed/` module wraps sentence-transformers with a TF-IDF fallback:

```
try:
    model = SentenceTransformer("all-MiniLM-L6-v2")
    embeddings = model.encode(texts)
except Exception:
    vectorizer = TfidfVectorizer(ngram_range=(1, 2))
    embeddings = vectorizer.fit_transform(texts).toarray()
```

This ensures the tools run in any environment. The TF-IDF fallback produces lower-quality embeddings but is sufficient for coarse alignment checking.

3 Analysis Tools

Paperscope includes 12 analysis tools organized into two groups: 6 consolidated from per-paper analysis scripts developed during the Coherence Dynamics research program, and 6 new tools built on the same infrastructure.

3.1 Citation Alignment

For each sentence containing a `\cite{}` command, citation alignment computes cosine similarity between the sentence embedding and all literature chunks, then checks whether the cited references appear among the top-k matches. The key metric is the **top-3 citation alignment rate**: the fraction of citation contexts where at least one cited reference appears in the three most similar literature chunks.

Formally, let $\mathbf{c}_i$ be the embedding of citation context i, and let $\mathbf{r}_{j,k}$ be the embedding of chunk k from reference j. For each context i citing references $\mathcal{K}_i$:

$$\text{aligned}_i = 1\left[\exists\, k \in \mathcal{K}_i : k \in \text{top-3}\left(\max_j \frac{\mathbf{c}_i \cdot \mathbf{r}_{k,j}}{|\mathbf{c}_i||\mathbf{r}_{k,j}|}\right)\right] \tag{1}$$

In practice, alignment rates of 60–80% are typical for well-cited papers. Contexts with low alignment are flagged for review — the citation may be correct but semantically indirect, or it may be misattributed.

3.2 Novelty Detection

Each claim extracted from the paper (bold assertions, numbered predictions) is embedded and compared against the full literature corpus. Claims whose maximum similarity to any literature chunk falls below a threshold (default $\tau = 0.4$) are flagged as **novel**. These are the claims reviewers will scrutinize most heavily, and where the author should ensure the strongest justification.

3.3 Reviewer Probes and Response Preparation

Two tools address reviewer preparation. **Reviewer probes** embed hypothetical reviewer questions and find the nearest literature chunks, revealing whether the bibliography contains material to respond to likely objections. **Reviewer response preparation** goes further: for each anticipated objection, it finds both (a) passages in the paper that address the objection and (b) literature that supports the response, creating a structured evidence map.

3.4 Self-Overlap and Cross-Paper Consistency

For authors with multiple publications, **self-overlap** detects high-similarity passages between the current paper and the author's other papers (threshold: cosine similarity > 0.75). This catches inadvertent self-plagiarism before reviewers do. **Cross-paper consistency** performs the inverse: it finds semantically similar passages across papers and flags them for manual review, since similar topics discussed differently may indicate contradictions.

3.5 Argument Flow

Argument flow tracks the paper's trajectory through embedding space paragraph by paragraph. Using PCA of paragraph embeddings, it computes cosine distance between consecutive paragraphs and flags **topic jumps** (distance $> \mu + 1.5\sigma$) and **semantic loops** (distant paragraphs with similarity > 0.85). This reveals structural issues: abrupt topic changes that need transitions, and repetitive passages that could be consolidated.

3.6 New Tools

Six additional tools extend the framework:

Abstract alignment embeds each section and the abstract, computing a coverage matrix. Sections with below-median similarity to the abstract are flagged as underrepresented.

Strength heatmap computes per-paragraph scores for citation support (max similarity to any literature chunk) and argument continuity (similarity to the preceding paragraph). Paragraphs with citation support < 0.3 are flagged as weak.

Journal targeting fetches recent abstracts from candidate journals via OpenAlex, embeds them, and ranks journals by centroid similarity to the paper. This replaces intuition-based journal selection with a semantic measure.

Revision diff embeds two versions of a paper, matches sections by title similarity, and measures the semantic shift per section. Optionally compares against a literature corpus to determine whether the revision moved the paper toward or away from the literature.

Related radar extracts keywords from the paper, searches OpenAlex, filters out papers already in the bibliography, embeds the remaining abstracts, and ranks by relevance. This finds missing related work before reviewers do.

Argument graph scans a directory of papers, extracts conclusions and premises from each, embeds them, and builds a directed dependency graph. An edge from paper A to paper B means A's conclusions support B's premises. This reveals the logical structure of a multi-paper research program.

4 Implementation

Paperscope is implemented in Python (3.8+) with minimal dependencies: `numpy` for array operations, `requests` for API calls, `sentence-transformers` for encoding (optional), and `PyMuPDF` for PDF text extraction. Visualization tools optionally use `matplotlib`, `scikit-learn`, and `networkx`.

All 15 commands are accessible via a single CLI entry point:

```
python3 -m paperscope analyze paper.tex --literature text/
python3 -m paperscope abstract-check paper.tex
python3 -m paperscope journal-fit paper.tex -j "BioSystems"
python3 -m paperscope revision-diff old.tex new.tex
python3 -m paperscope related paper.tex
```

Analysis commands produce both structured JSON output and human-readable console summaries. The JSON output is designed for consumption by AI coding assistants: Claude Code or Codex can read the results, identify issues, and suggest fixes within the same conversation.

External APIs (CrossRef, OpenAlex, Unpaywall) are used in their free tiers with polite rate limiting. No API keys are required; only an email address for courtesy identification.

5 Usage Context

Paperscope was developed for the Coherence Dynamics research program (https://coherencedynamics.com), which has produced 30+ papers across neuroscience, biology, physics, and mathematics. The toolkit manages ∼1,500 references across 175 `.tex` files.

The typical workflow integrates Paperscope with an AI coding assistant:

1. Write the paper (with AI assistance for drafting, derivations, and literature search).
2. Run `analyze` to check citation alignment, novelty, and argument strength.
3. The AI assistant reads the JSON output, identifies weak citations or unsupported claims, and suggests specific fixes.

4. Run `abstract-check` before submission to verify coverage.
5. Run `journal-fit` to select the target venue.
6. After revision, run `revision-diff` to verify that changes addressed reviewer concerns and measure the direction of semantic shift.

This workflow treats the AI assistant as a collaborator that can read structured analysis output and act on it, rather than as a black box that generates text.

6 AI Workflow Disclosure

This paper and the Paperscope software were developed with extensive use of AI tools:

- **Claude Opus 4** (Anthropic, via Claude Code): Primary development environment. Used for code generation, refactoring, architecture design, and drafting this paper. All code was reviewed and validated by the human author.
- **GPT-4.5 / Codex** (OpenAI): Used for cross-validation of code logic and fact-checking of API integration details.

The human contribution consists of: problem identification (recognizing the need for embedding-based citation analysis), architectural decisions (three-layer design, TF-IDF fallback, CLI-first interface), tool selection (which analyses are actually useful during paper writing), and validation against real manuscripts.

7 Related Work

Several tools address parts of the academic writing pipeline:

Semantic Scholar [Kinney et al., 2023] provides a citation graph and paper recommendations but does not operate at the intra-paper level (individual sentences and claims). **scite.ai** classifies citations as supporting, contrasting, or mentioning, but requires their proprietary database and does not support custom literature corpora. **Connected Papers** visualizes citation networks but does not analyse the content of citing sentences. **citation.js** [Willighagen, 2019] manages bibliography formatting but does not perform semantic analysis.

Paperscope differs in operating at the sentence-chunk level rather than the paper level, working with the author's own literature corpus rather than a proprietary database, and producing structured output designed for AI assistant consumption.

8 Availability

Paperscope is available at `https://github.com/todd866/paperscope` under the MIT license. Requirements: Python 3.8+, `numpy`, `requests`, and optionally `sentence-transformers` for neural embeddings. Installation: `pip install -r requirements.txt`.

References

CrossRef. CrossRef REST API, 2024. URL https://www.crossref.org/documentation/retrieve-metadata/rest-api/. Accessed March 2026.

Rodney Kinney, Chloe Anastasiades, Russell Authur, Iz Beltagy, Jonathan Bragg, Alexandra Buber, Jared Callaham, Teresa Cash-Padgett, Davide Castelvecchi, Arman Choudhury, Doug Downey, et al. The Semantic Scholar Open Data Platform. *arXiv preprint arXiv:2301.10140*, 2023. doi: 10.48550/arXiv.2301.10140.

Jason Priem, Heather Piwowar, and Richard Orr. OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. *arXiv preprint arXiv:2205.01833*, 2022. doi: 10.48550/arXiv.2205.01833.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019. doi: 10.18653/v1/D19-1410.

Lars G. Willighagen. Citation.js: a format-independent, modular bibliography tool for the browser and command line. *PeerJ Computer Science*, 5:e214, 2019. doi: 10.7717/peerj-cs.214.